# Seminar 2

Last time: start from the **theorem**, leave holes `?_`, **fill them in**.

Today: the same loop, but you get to **name the moves**. Nothing new is being proved — the same terms are being written a different way.

# last week: implS

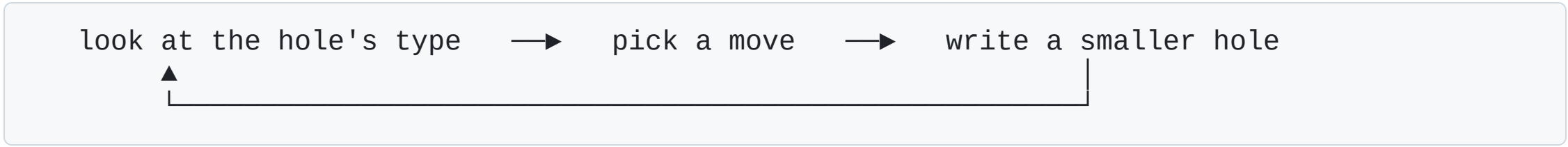
```
theorem implS {A B C : Prop} :
    (A → B → C) → (A → B) → A → C :=
  fun f g a => f a (?_)
```

applied `f a` — the hole must be a `B`

```
f : A → B → C
g : A → B
a : A
⊢ B
```

**You wrote a term. But you did not write it in one go — you wrote a hole, read what Lean said, and refined.**

# You were already running a loop




The **file** was static. The **process** was not.

Lean kept the state — the hypotheses in scope and the type of the hole — and handed it back to you after every edit.

Tactics are that loop, with the moves given names.

# The same proof, twice

```
theorem iff_and_congr (hab : A ↔ B) (hcd : C ↔ D) : A ∧ C ↔ B ∧ D :=
  ⟨ fun h1 => ⟨ hab.mp h1.left , hcd.mp h1.right ⟩ ,
    fun h2 => ⟨ hab.mpr h2.left , hcd.mpr h2.right ⟩ ⟩
```

```
theorem iff_and_congr (hab : A ↔ B) (hcd : C ↔ D) : A ∧ C ↔ B ∧ D := by
  constructor
  · intro h
    exact ⟨hab.mp h.left, hcd.mp h.right⟩
  · intro h
    exact ⟨hab.mpr h.left, hcd.mpr h.right⟩
```

# And you can work forwards

> "From `hA` and `hr` I get `role B = .knight`. Put it aside; I will use it in a moment."

```
have hb : role B = .knight := hA.mp hr
```

```
hA : role A = .knight ↔ role B = .knight
hB : role B = .knight ↔ role A = .knave
hr : role A = .knight
hb : role B = .knight      -- new
⊢ False
```

# Underneath, it is a term

```
theorem implModusPonens {A B : Prop} : (A → B) ∧ A → B := by
  intro h
  exact h.left h.right

#print implModusPonens
```

```
theorem Seminar02.implModusPonens : ∀ {A B : Prop}, (A → B) ∧ A → B :=
fun {A B} h => h.left h.right
```

# But not always the term you'd write

```
theorem andOrDistrib (h : A ∧ (B ∨ C)) : (A ∧ B) ∨ (A ∧ C) := by
  obtain ⟨a, hbc⟩ := h
  cases hbc with
  | inl b => left;  exact ⟨a, b⟩
  | inr c => right; exact ⟨a, c⟩
```

```
fun {A B C} h =>
  And.casesOn h fun a hbc =>
    Or.casesOn (motive := fun t => hbc = t → A ∧ B ∨ A ∧ C) hbc
      (fun b h => Or.inl ⟨a, b⟩) (fun c h => Or.inr ⟨a, c⟩) (Eq.refl hbc)
```

# The dictionary — using and building

| you wrote (hw01) | you can now say |
|---|---|
| `?_` | one goal in the goal state |
| `fun x => ?_` | `intro x` |
| the finished term `e` | `exact e` |
| `f ?_ ?_` | `apply f` |
| `⟨?_, ?_⟩`, `And.intro`, `Iff.intro` | `constructor` |
| `Or.inl ?_` / `Or.inr ?_` | `left` / `right` |
| `match h with \| .inl a => ?_ \| .inr b => ?_` | `cases h with ...` / `rcases h with a \| b` |
| `match Classical.em A with ...` | `by_cases hA : A` |
| `fun ⟨a, b⟩ => ?_`, `h.left`, `h.right` | `obtain ⟨a, b⟩ := h` |
| naming a subterm you reuse | `have hb : B := ...` |
| `False.elim ?_` | `exfalso` |

# New this week: rewriting and quantifiers

| tactic | what it does |
|---|---|
| `cases hr : role x` | one goal per constructor of `Role`; `hr` records which |
| `rfl` | closes `a = b` when both sides compute to the same value |
| `rw [h]` | `h : a = b`, so replace `a` by `b` in the goal |
| `rw [← h]` | the same rewrite, right to left |
| `rw [h] at h'` | rewrite inside a hypothesis instead of the goal |
| `intro x` | `∀ x, P x` is a function type — the same tactic as for `→` |
| `exact ⟨x, prf⟩` | `∃ x, P x` is a pair: a witness, and a proof about it |
| `obtain ⟨x, hx⟩ := h` | take that pair apart |

# Round 1 — translate

`code/Course/Seminar02.lean`, round 1. Four of last week's exercises, with your term proof in the comment above each. **Write the tactic proof.**

```
andAssocBack    : A ∧ (B ∧ C) → (A ∧ B) ∧ C
iffNotCongr     : (A ↔ B) → (¬A ↔ ¬B)
noContradiction : ¬(A ∧ ¬A)
notNotEm        : ¬¬(A ∨ ¬A)
```

# andAssocBack

```
theorem andAssocBack {A B C : Prop} (h : A ∧ (B ∧ C)) :
    (A ∧ B) ∧ C := by
  obtain ⟨a, b, c⟩ := h
  exact ⟨⟨a, b⟩, c⟩
```

after `obtain`

```
a : A
b : B
c : C
⊢ (A ∧ B) ∧ C
```

after `exact ⟨⟨a, b⟩, c⟩`

No goals

**`obtain` names the pieces once; hw01 reached in with `h.left` and `h.right.left` instead.**

# iffNotCongr

```
theorem iffNotCongr {A B : Prop} (hab : A ↔ B) : ¬A ↔ ¬B := by
  constructor
  · intro na b
    exact na (hab.mpr b)
  · intro nb a
    exact nb (hab.mp a)
```

after `constructor` — 2 goals

```
⊢ ¬A → ¬B
⊢ ¬B → ¬A
```

after both bullets

No goals

**`constructor` on an `↔` is the `⟨_, _⟩` you wrote by hand; the dots pick which half you are on.**

# noContradiction

```
theorem noContradiction {A : Prop} : ¬(A ∧ ¬A) := by
  intro h
  exact h.right h.left
```

after `intro h`

```
h : A ∧ ¬A
⊢ False
-- ¬X is X → False, so the goal
-- of a negation is always False
```

after `exact h.right h.left`

No goals

**A negation goal starts with `intro`, like any other function goal. `¬` carries no tactics of its own.**

# notNotEm

```
theorem notNotEm {A : Prop} : ¬¬(A ∨ ¬A) := by
  intro h
  ?_
```

```
h : ¬(A ∨ ¬A)
⊢ False
-- h : (A ∨ ¬A) → False
-- the only move: feed it an A ∨ ¬A
```

# notNotEm

```
theorem notNotEm {A : Prop} : ¬¬(A ∨ ¬A) := by
  intro h
  apply h
  right
  ?_
```

`apply h`, then pick a side

```
h : ¬(A ∨ ¬A)
⊢ A ∨ ¬A          -- after apply h
⊢ ¬A              -- after right
-- left would need an A, and
-- we have no A. So: right.
```

# notNotEm

```
theorem notNotEm {A : Prop} : ¬¬(A ∨ ¬A) := by
  intro h
  apply h
  right
  intro a
  apply h
  left
  exact a
```

now we have an `A` — use `h` again

```
a : A
h : ¬(A ∨ ¬A)
⊢ False   -- after intro a
⊢ A ∨ ¬A  -- after apply h
⊢ A       -- after left
```

No goals

# `#print notNotEm`

```
theorem Seminar02.notNotEm : ∀ {A : Prop}, ¬¬(A ∨ ¬A) :=
fun {A} h => h (Or.inr fun a => h (Or.inl a))
```

# Round 2 — back-translate

Round 2 of the file. Two tactic proofs are **given**.

For each: predict what `#print` shows, write the term yourself underneath, then run it and compare.

```
implModusPonens : (A → B) ∧ A → B          -- intro / exact
andOrDistrib    : A ∧ (B ∨ C) → (A ∧ B) ∨ (A ∧ C)   -- obtain / cases
```

# Round 2 — the answers

```
-- implModusPonens: exactly what you would type
fun {A B} h => h.left h.right

-- andOrDistrib: not what you would type
fun {A B C} h =>
  And.casesOn h fun a hbc =>
    Or.casesOn (motive := fun t => hbc = t → A ∧ B ∨ A ∧ C) hbc
      (fun b h => Or.inl ⟨a, b⟩) (fun c h => Or.inr ⟨a, c⟩) (Eq.refl hbc)
```

# Round 3 — where the term is not worth writing

```
flipFlipFlip    : flip (flip (flip r)) = flip r     -- cases, rfl
roleChain3      : role A = role B → role B = role C → role A = role C
roleChain3'     : the same, backwards and at a hypothesis
forallImpForall : (∀ x, P x → Q x) → (∀ x, P x) → ∀ x, Q x
existsSwap      : (∃ x, ∃ y, P x y) → ∃ y, ∃ x, P x y
puzzleMutualAccusation  -- last week's puzzle, new encoding
```

# flipFlipFlip

```
def Role.flip : Role → Role
  | .knight => .knave
  | .knave  => .knight

theorem flipFlipFlip (r : Role) :
    Role.flip (Role.flip (Role.flip r)) = Role.flip r := by
  cases r <;> rfl
```

after `cases r` — 2 goals

```
⊢ flip (flip (flip knight))
    = flip knight
⊢ flip (flip (flip knave))
    = flip knave
-- both sides compute
```

after `rfl` on each

No goals

**Two constructors, so `cases r` opens two goals — and `<;>` runs `rfl` on every one of them.**

# The rewrites

```
theorem roleChain3 (h₁ : role A = role B) (h₂ : role B = role C) :
    role A = role C := by
  rw [h₁, h₂]

theorem roleChain3' (h₁ : role A = role B) (h₂ : role B = role C) :
    role A = role C := by
  rw [← h₁] at h₂
  exact h₂
```

the goal, rewritten twice

```
⊢ role A = role C
⊢ role B = role C   -- rw [h₁]
⊢ role C = role C   -- rw [h₂]
```

No goals

**`rw [h]` rewrites the goal; `← h` reverses the direction, `at h'` aims it at a hypothesis.**

# The quantifiers, with no new tactics

```
theorem forallImpForall {α : Type} (P Q : α → Prop) :
    (∀ x, P x → Q x) → (∀ x, P x) → ∀ x, Q x := by
  intro h hp x
  exact h x (hp x)

theorem existsSwap {α β : Type} (P : α → β → Prop) :
    (∃ x, ∃ y, P x y) → ∃ y, ∃ x, P x y := by
  intro h
  obtain ⟨x, y, hxy⟩ := h
  exact ⟨y, x, hxy⟩
```

three `intro`s, one of them a `∀`

```
h  : ∀ x, P x → Q x
hp : ∀ x, P x
x  : α
⊢ Q x
```

# puzzleMutualAccusation

```
theorem puzzleMutualAccusation (A B : Islander)
    (hA : Says A (role B = .knight)) (hB : Says B (role A = .knave)) :
    claim% answerMutualAccusation [A, B] := by
  unfold Says at hA hB
  cases hr : role A with
  | knight =>
    have hna : role A = .knave := hB.mp (hA.mp hr)
    rw [hr] at hna
    cases hna
  | knave =>
    have hka : role A = .knight := hA.mpr (hB.mpr hr)
    rw [hr] at hka
    cases hka
```

**The answer is `impossible`, so the goal is `False`. Two constructors, two cases; `rw [hr]` turns each `have` into `knight = knave`, and `cases` on that closes the goal.**

# The same puzzle, last week

```
-- hw01: an islander was a Prop, and the split came from an axiom
match Classical.em A with
| Or.inl a  => (hB.mp (hA.mp a)) a
| Or.inr na => na (hA.mpr (hB.mpr na))
```

```
-- today: an islander is a value, and the split comes from the data
cases hr : role A with
| knight => ...
| knave  => ...
```

```
#print axioms puzzleMutualAccusation
-- 'Seminar02.puzzleMutualAccusation' does not depend on any axioms
```

**The case split now comes from `Role` having exactly two constructors. No excluded middle, no axioms.**

# What to take away

- A goal **is** a hole; `intro` / `exact` / `apply` are edits to it.
- `#print` shows what your tactics wrote — often not the term you would write.
- `∀` is `→` with a name, `∃` is `∧` with a witness — no new tactics.
- A finite type's case split comes from the **data**, not from `Classical.em`.
- `rw [h]` rewrites the goal; `← h` reverses it, `at h'` aims it elsewhere.
- Everything else today was a **renaming** of something you already did.